

Chapter 1 — Introduction to datasets

Define a dataset as curated records and attributes prepared for a purpose

Learning objectives

- Define what counts as a dataset (vs raw logs)
- Distinguish records (rows) from attributes (columns)
- Recognize the same idea in CSV and JSON forms

What is a dataset?

- Curated collection of data for analysis or decisions
- Typically organized as records and attributes
- Prepared for a purpose: scope, structure, documentation
- Basic unit for ML, statistics, and business intelligence

Records and attributes

- **Record** — one observation (row, instance)
- **Attribute** — one property of that observation
(column, field)
- Sales example: one sale = one record; Date, Product,
Price = attributes
- Same pattern in healthcare, sensors, and other
domains

Example 1.1 — Sample CSV sales data

```
$ cd modules/chapter1/example1/ && bash run.sh
```

Example 1.4 — Housing prices in JSON

```
$ cd modules/chapter1/example4/ && bash run.sh
```

Takeaways

- A dataset is curated for a purpose, not just a dump of bytes
- Records + attributes are the core vocabulary
- Form varies (CSV, JSON, images, text); the goal stays insight

Learning objectives

- Contrast structured, unstructured, and semi-structured data
- Give a typical format and example for each structural type
- Distinguish static from dynamic datasets

Two classification axes

- **Structure:** structured / unstructured / semi-structured
- **Temporal behavior:** static vs dynamic
- Structure affects how you store, query, and analyze
- Choosing the right representation is a prerequisite for reliable analysis

Structured datasets

- Fixed schema: rows and columns, consistent types
- Common stores: spreadsheets, relational tables, CSV
- Easy to filter, join, and aggregate
- Example: customer table (ID, Name, Email, ...)

Unstructured datasets

- No fixed row–column schema
- Text, images, audio, video
- Insights often need NLP or computer vision
- Example: social posts, X-rays, voice recordings

Semi-structured datasets

```
$ cd modules/chapter1/example7/ && bash run.sh
```

Static vs dynamic

- **Static:** snapshot that changes rarely (archive census extract)
- **Dynamic:** updates continuously (sensor streams, click logs)
- Same entity can appear in both forms over its lifecycle
- Temporal behavior drives refresh, versioning, and monitoring needs

Takeaways

- Structure and time are independent axes
- Structured data is easiest to query; unstructured needs extraction
- Semi-structured sits between: flexible keys, still machine-readable

Learning objectives

- Name common formats for tabular, hierarchical, and geospatial data
- Explain when CSV, SQL, HDF5, or GeoJSON is a fit
- Link format choice to downstream tooling

Why format matters

- Format is how structure is serialized on disk
- Wrong format creates friction in the pipeline
- Match format to structure, volume, and tools
- Complements the type taxonomy from the previous clip

Common formats at a glance

- **CSV** — simple tabular exchange
- **SQL tables** — queryable structured stores
- **JSON / GeoJSON** — nested or spatial features
- **HDF5** — large scientific / array datasets

Example 1.8 — SQL format

```
$ cd modules/chapter1/example8/ && bash run.sh
```

Example 1.9 — HDF5

```
$ cd modules/chapter1/example9/ && bash run.sh
```

Choosing a format

- Start from structure (tabular vs nested vs spatial)
- Consider size, update rate, and who will query it
- Prefer formats your stack already supports well
- Document the choice in metadata (next quality topics)

Takeaways

- Format = serialization of structure
- CSV and SQL dominate everyday tables
- HDF5 and GeoJSON serve specialized shapes and scales

Learning objectives

- List five quality dimensions from the chapter
- Diagnose accuracy, completeness, and consistency defects
- Explain why metadata and documentation matter

Five quality dimensions

- **Accuracy** — values reflect ground truth closely enough
- **Completeness** — needed records and attributes are present
- **Consistency** — uniform formats, units, definitions
- **Relevance** — data answers the intended question
- **Timeliness** — data is sufficiently up to date

Example 1.11 — Incorrect transaction

```
$ cd modules/chapter1/example11/ && bash run.sh
```

Completeness and consistency

- Missing age or diagnosis fields bias clinical models
(eg:1.12)
- Mixed date formats break chronological sorting
(eg:1.13)
- Defects interact: complete yet inaccurate still
fails the task
- Systematic repair is deferred to later chapters

Metadata and documentation

```
$ cd modules/chapter1/example14/ && bash run.sh
```

Takeaways

- Quality is multi-dimensional and task-relative
- Spot accuracy, missingness, and format clashes early
- Metadata + docs make datasets usable beyond their author

Learning objectives

- State three early questions exploration should answer
- Match common tools to dataset size and structure
- Outline a first-pass profiling pattern (summaries + visuals + missingness)

Why explore first?

- Turns an unfamiliar file into a mental model
- Surfaces seasonal patterns, outliers, missing fields
- Shapes which quality issues to prioritize
- Does not replace formal cleaning—reveals where it is needed

Three early questions

- What is in the table?
- What looks suspicious?
- What analysis goals remain feasible given defects?

Tools for exploration

- Spreadsheets — small tables, quick looks
- SQL — filter and aggregate relational stores
- pandas — reproducible profiling scripts
- Tableau / Power BI — shared interactive views

Example 1.20 — Household incomes EDA

```
$ cd modules/chapter1/example20/ && bash run.sh
```

Takeaways

- Explore before modeling to set expectations
- Tool choice follows size, structure, and audience
- Document what you inspected and what you will fix

Learning objectives

- Map sectors to typical datasets and analytical goals
- Trace the shared lifecycle: define, assess, explore, act
- Outline the customer-churn case study steps

Applications across sectors

- Retail: purchase histories → segmentation, forecasts
- Healthcare: EHRs and wearables → risk and monitoring
- Finance: ledgers → fraud and credit scoring
- Education and government: logs and statistics →
intervention and planning

Example 1.21 — Purchase history

```
$ cd modules/chapter1/example21/ && bash run.sh
```

Case study: customer churn

- Collect and join customer behavior features
- Explore and clean before modeling
- Train a classifier; score risk of cancellation
- Act with retention offers guided by feature importance

Example 1.31 — Churn model sketch

```
$ cd modules/chapter1/example31/ && bash run.sh
```

Takeaways

- Domain vocabulary changes; the lifecycle does not
- Exploration and quality feed every serious application
- Churn shows define → prepare → model → intervene

Learning objectives

- Explain why management matters after models ship
- List core management practices (versioning, metadata, access, catalog)
- Name privacy and security controls that protect sensitive tables

Why management matters

- Quiet work: access, versions, backups, retention
- Traceability for audits and collaboration
- Prevents undocumented preprocessing from poisoning models
- Protects the investment in collection and quality

Best practices (overview)

- Version control for evolving extracts
- Cleaning and transformation with clear schemas
- Metadata documentation and data catalogs
- Storage, backups, and role-based access control

Privacy and security

- Encryption at rest and in transit
- Anonymization / de-identification for sharing
- MFA and role-based access
- Compliance (e.g., GDPR, HIPAA) plus audit logging

Looking ahead

- This chapter: what datasets are and how to use them responsibly
- Next: Chapter 2 — how records are collected
- Management depth continues in later chapters (privacy, docs, storage)

Takeaways

- Management is not an optional add-on
- Versioning + metadata + access must work together
- Protection begins when records are first stored